

Lecture 2

Persistent Data Structures

CSEN 1038 Advanced Data Structures and Algorithms

Table of contents

1. Introduction
2. Types of persistence
3. Different techniques for implementation
4. Persistent Segment Tree
5. Historical Sum Problem

Introduction

- Motivation: keep old versions while supporting updates
- Versioning and time-travel queries

Types of persistence

Types of persistence

- Partial persistence: updates on latest version; queries on any version
- Full persistence: updates and queries on any version
- Confluent persistence: merge versions

Different techniques for implementation

Different techniques for implementation

- **Fat nodes**

- Each node is expanded to store a history of all its previous values and version stamps in a list or array.

- **Path copying**

- When a node is modified, a new version of that node and all its ancestors up to the root are created.

- **Node-copying**

- A hybrid approach that uses a fixed number of modification boxes per node, copying the node only when the boxes are full.

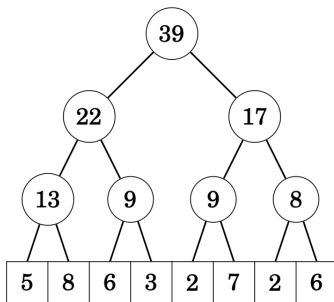
Persistent Segment Tree

Example: persistent segment tree

- Build version 0 from the initial array
- Each update creates a new root (copy path to the leaf)
- Query any version in $O(\log n)$

Persistent Segment Tree

For example, the segment tree



is stored as follows:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
39	22	17	13	9	9	8	5	8	6	3	2	7	2	6

Historical Sum Problem

Historical Sum Problem: Problem Framework

This problem adds a temporal dimension to the standard range-sum query by requiring the system to manage multiple versions of the same array:

1. **Initialization:** Start with an initial array A_0 of size n (“Version 0”).
2. **Support updates:** When an update occurs (e.g., changing $A[i]$ to x), the system does not overwrite old data. Instead, it creates a new version $(A_1, A_2, \dots, A_m)$, where each version represents the state of the array after that modification.
3. **Support queries:** Answer queries for a range $[L, R]$ on a specific version index v by computing

$$\sum_{i=L}^R A_v[i].$$

Questions?

- https://cp-algorithms.com/data_structures/segment_tree.html#preserving-the-history-of-its-values-persistent-segment-tree
- <https://www.youtube.com/watch?v=EQVtLGEOIxQ&list=PLc02D4EoVYQDzOwJKj3g-KIRYgPK08Mvm&index=10>